

Code Review Report

Language: python | Review mode: openai | Model: llama-3.3-70b-versatile | Generated: 2026-08-13 13:49 UTC

Overall Assessment

The code review has identified 6 security findings, with 5 critical and 1 medium severity issues. These findings pose significant security risks, including hardcoded sensitive information, SQL injection vulnerabilities, and use of eval and exec functions with user input.

Merge readiness: Not Ready

Code health: 12/100

Findings Summary

Total findings: 6

Risk score: 22

Severity breakdown: Low 0, Medium 1, High 0, Critical 5

Type	Severity	Line	Message	Recommendation
Security	Critical	1	Hardcoded sensitive information	Store sensitive information securely using environment variables or a secrets manager
Security	Critical	2	Hardcoded API key	Store API keys securely using environment variables or a secrets manager
Security	Critical	5	SQL injection vulnerability	Use parameterized queries or prepared statements to prevent SQL injection
Security	Critical	7	Use of eval function with user input	Avoid using eval function with user input, use safer alternatives like ast.literal_eval
Security	Medium	9	Use of exec function	Avoid using exec function, use safer alternatives like subprocess.run with a list of arguments
Security	Critical	11	Use of subprocess.run with shell=True	Avoid using shell=True, use a list of arguments to prevent shell injection

Remediation Roadmap

Finding	Severity	Impact	Suggested Action
Security	Critical	Hardcoded sensitive information	Store sensitive information securely using environment variables or a secrets manager
Security	Critical	Hardcoded API key	Store API keys securely using environment variables or a secrets manager
Security	Critical	SQL injection vulnerability	Use parameterized queries or prepared statements to prevent SQL injection

Finding	Severity	Impact	Suggested Action
Security	Critical	Use of eval function with user input	Avoid using eval function with user input, use safer alternatives like ast.literal_eval
Security	Critical	Use of subprocess.run with shell=True	Avoid using shell=True, use a list of arguments to prevent shell injection
Security	Medium	Use of exec function	Avoid using exec function, use safer alternatives like subprocess.run with a list of arguments

Source Code Snapshot

```
password = "admin123"

api_key = "123456789"

name = input()

query = "SELECT * FROM users WHERE name='" + name + "'"

eval(input())

exec("print('Hello')")

import subprocess

subprocess.run("dir", shell=True)
```